

Algoritmos Genéticos: listas simplemente ligadas y aterrizaje evolutivo

Jorge Andrés Martínez Santos | Marián Garcés Delgado | Mariana Sinisterra
Politécnico Colombiano Jaime Isaza Cadavid — Medellín, Colombia

1. Descripción general

Punto 1 — Listas simplemente ligadas. El problema consiste en encontrar una cadena de operaciones que transforme un valor inicial de cero en un número objetivo definido por el usuario. Cada posición de la cadena puede contener una de tres operaciones: sumar uno (+1), sumar dos (+2) o multiplicar por dos ($\times 2$). Por ejemplo, para alcanzar el objetivo 11 se puede utilizar la secuencia +1, $\times 2$, $\times 2$, +1, $\times 2$, +1. Aunque una persona puede resolver una instancia pequeña por prueba y error, el número de cadenas posibles crece exponencialmente al aumentar la longitud del cromosoma.

La finalidad del algoritmo genético es optimizar la cercanía entre el resultado producido por cada cadena y el objetivo solicitado. La particularidad del ejercicio es que cada cromosoma se implementa como una lista simplemente ligada: cada nodo almacena una operación y una referencia al siguiente nodo. La población también se almacena mediante nodos enlazados. La aplicación permite modificar población, generaciones, cruce, mutación y elitismo, y conserva en una tabla todas las cadenas evaluadas para observar cómo se construye el mejor resultado.

Punto 2 — Aterrizaje. El segundo problema consiste en evolucionar el control de una nave que comienza en el aire y debe llegar a una pista. Cada individuo contiene doce acciones; una acción combina un giro (-1, 0 o 1) y la decisión de activar o no el propulsor. La simulación aplica gravedad, empuje, velocidad y ángulo durante ocho instantes por cada acción.

En este punto se optimizan simultáneamente la cercanía al centro de la pista, las velocidades horizontal y vertical, y la inclinación de la nave. No basta con tocar el suelo: para considerarse un aterrizaje, debe caer sobre la pista, hacerlo lentamente y conservar un ángulo seguro. La interfaz permite cambiar población, cruce, mutación, elitismo y velocidad de visualización.

2. Trabajos relacionados

Holland [1] formuló las bases de los algoritmos genéticos y explicó cómo una población codificada puede evolucionar mediante selección y recombinación. Su propuesta sustenta el ciclo utilizado en este trabajo: crear individuos, evaluarlos, seleccionar progenitores y producir nuevas generaciones hasta cumplir una condición de parada.

Goldberg [2] presentó aplicaciones prácticas de los algoritmos genéticos en búsqueda y optimización, además de analizar la presión selectiva, el cruce y la mutación. Estos conceptos se relacionan directamente con la selección por torneo y con las probabilidades configurables de cruce y mutación usadas para evolucionar las cadenas de operaciones.

Mitchell [3] destacó que la representación del cromosoma y la función de aptitud determinan gran parte del comportamiento de un algoritmo genético. En esta aplicación, la representación no es un arreglo tradicional, sino una lista simplemente ligada. Goodrich, Tamassia y Goldwasser [4] describen esta estructura como una secuencia de nodos conectados mediante una única referencia, principio empleado tanto en los cromosomas como en la población.

Michalewicz y Fogel [5] estudiaron el uso de metaheurísticas en problemas donde una solución es una secuencia de decisiones. Ese enfoque se relaciona con el punto de aterrizaje, porque el cromosoma no representa una posición final aislada, sino un programa de acciones de giro y empuje que produce una trayectoria completa.

3. Función objetivo

Punto 1. La función objetivo asigna mayor aptitud a las cadenas cuyo resultado se encuentra más cerca del valor objetivo. Sea O el objetivo indicado por el usuario y R(c) el resultado de ejecutar, desde cero, todas las operaciones del cromosoma c. Se resta la distancia absoluta a una base de 100 y se establece 1 como valor mínimo para que todos los individuos puedan participar en la selección. Cuando R(c) es igual a O, el fitness alcanza su máximo de 100.

$$f(c) = \max(1, 100 - |O - R(c)|)$$

Ecuación 1. Función objetivo del punto 1.

Punto 2. La aptitud de la nave combina cuatro términos continuos: distancia horizontal al centro de la pista, velocidad vertical, velocidad horizontal y ángulo. Además, suma bonificaciones por tocar el suelo, hacerlo sobre la pista y cumplir completamente las condiciones de aterrizaje. Por tanto, una trayectoria puede mejorar gradualmente antes de conseguir un aterrizaje.

$$F = 4000/(1 + 0,15|x - c|) + 1500/(1 + |vy|) + 600/(1 + |vx|) + 400/(1 + 2|a|) + B$$

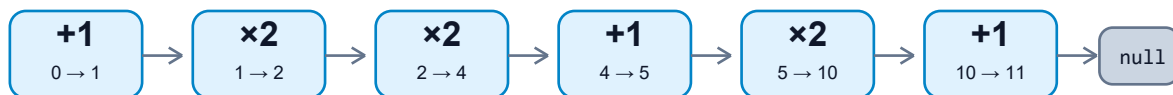
Ecuación 2. Función objetivo del punto 2.

4. Definición de población

Punto 1. La población es una ListaPoblacion formada por objetos NodoIndividuo. Cada nodo guarda un Individuo y la referencia al siguiente. Un individuo contiene su ListaOperaciones, el resultado obtenido y el fitness. Con la configuración inicial se generan 40 cadenas aleatorias, aunque el usuario puede elegir entre 2 y 200 individuos desde la interfaz.

El cromosoma es una ListaOperaciones. Cada NodoOperacion contiene uno de los genes válidos (+1, +2 o ×2) y un enlace siguiente; el último enlace vale null. La Figura 1 presenta una cadena de seis nodos que transforma cero en once. Esta representación obliga a recorrer la lista desde la cabeza para evaluar, cruzar, mutar o mostrar el individuo.

cabeza

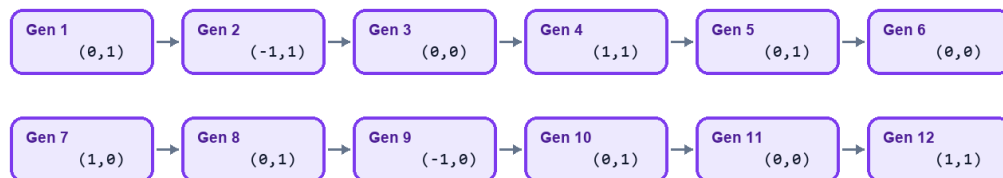


Resultado = 11 Fitness = 100

Figura 1. Cromosoma definido para el punto 1.

Punto 2. La población inicial contiene 60 naves. Cada cromosoma es una secuencia de doce genes y cada gen almacena dos valores: rotación (-1 izquierda, 0 sin giro o 1 derecha) y propulsión (0 apagada o 1 encendida). La simulación conserva para cada individuo su trayectoria, posición final, velocidades, ángulo, estado y fitness.

La Figura 2 representa un cromosoma de control. Los genes se ejecutan de izquierda a derecha y cada uno se mantiene durante ocho pasos físicos. De este modo, el orden de las acciones modifica completamente la ruta de la nave y el instante en que utiliza el combustible.



Cada gen = (rotación, propulsión)

Figura 2. Cromosoma definido para el punto 2.

5. Estrategia de cruce y mutación

Punto 1. El cruce utiliza un punto de corte aleatorio situado entre el primer y el último nodo. El hijo recibe los nodos anteriores al corte del padre y los nodos restantes de la madre. Las operaciones se copian en nodos nuevos; de esta manera, padres e hijos no comparten referencias y una modificación posterior no altera accidentalmente otro cromosoma. El cruce solo se aplica cuando un número aleatorio cumple la probabilidad configurada.

La mutación selecciona una posición aleatoria y sustituye su operación por otra diferente entre +1, +2 y $\times 2$. Esta variación permite introducir información que no estaba presente en los padres y ayuda a escapar de resultados estancados. Antes de crear hijos se conservan los N mejores individuos mediante elitismo; los demás progenitores se eligen por torneos de tres candidatos.

Punto 2. El cruce de las naves también utiliza un punto de corte: copia las primeras acciones de un progenitor y las restantes del otro. Los padres se escogen mediante torneo entre individuos de la parte superior de la población ordenada por fitness. Los mejores individuos indicados por el elitismo pasan completos a la siguiente generación.

La mutación se evalúa de forma independiente sobre cada gen del control. Cuando se cumple la probabilidad P_m , se genera una nueva combinación de giro y propulsión. Esta mutación puntual puede cambiar la orientación o el empuje en un momento específico sin reemplazar toda la trayectoria.

6. Aplicación construida

Punto 1. La aplicación fue construida en Java con programación orientada a objetos. Las estructuras `NodoOperacion`, `ListaOperaciones`, `NodoIndividuo` y `ListaPoblacion` fueron implementadas manualmente. La interfaz utiliza Java Swing y ejecuta la evolución en un `SwingWorker` para evitar que la ventana se bloquee. El usuario puede cambiar objetivo, nodos, población, generaciones, cruce, mutación, elitismo y pausa. El panel superior dibuja la mejor lista y la tabla inferior registra todas las cadenas creadas, su resultado y fitness. En la Figura 3 se presenta la aplicación durante una ejecución.



Figura 3. Aplicación construida para el punto 1.

Punto 2. La aplicación de aterrizaje fue construida con HTML, CSS y JavaScript. Dos elementos canvas dibujan la simulación y la gráfica de fitness. El panel lateral contiene controles para población, P_c , P_m , elitismo y velocidad. Durante la ejecución se presentan la generación, el estado de la mejor nave, el número de aterrizajes y el mayor fitness. En la Figura 4 se muestra la interfaz de esta aplicación.

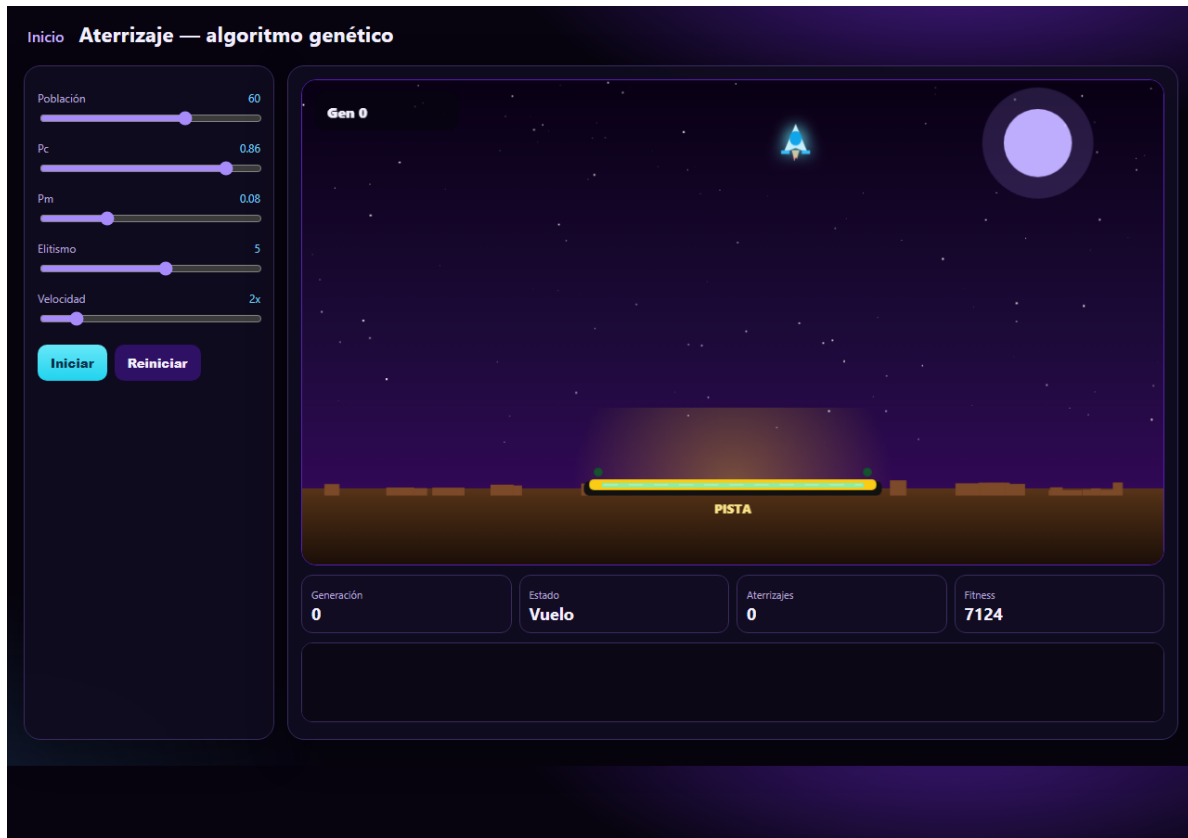


Figura 4. Aplicación construida para el punto 2.

En la siguiente URL se puede acceder al código de la aplicación: <https://github.com/GeorgeAMS/AlgoritmoGenetico>

7. Conclusiones

El algoritmo genético permite encontrar cadenas cercanas o iguales al objetivo sin enumerar manualmente todas las combinaciones. La calidad de la búsqueda depende del equilibrio entre conservación y diversidad: un elitismo alto protege soluciones buenas, mientras que el cruce y la mutación exploran nuevas cadenas.

La implementación con listas simplemente ligadas demuestra que un cromosoma no tiene que almacenarse necesariamente en un arreglo. Sin embargo, acceder a una posición requiere recorrer los enlaces anteriores, por lo que la estructura hace visibles tanto las ventajas de una representación dinámica como su coste de recorrido.

Mostrar todas las cadenas de todas las generaciones facilita la explicación del proceso evolutivo. La aplicación no presenta únicamente la solución final: permite comparar individuos, identificar élites y observar el efecto de cambiar población, mutación y cruce.

En el segundo punto, una función de fitness gradual resulta necesaria porque los aterrizajes válidos son poco frecuentes al inicio. Premiar la proximidad, la estabilidad y la reducción de velocidad guía a la población hacia una solución antes de que alguna nave aterrice.

Los dos puntos comparten selección, cruce, mutación y elitismo, pero demuestran que la representación y el fitness deben diseñarse según el problema. Una lista de operaciones busca un valor exacto; una secuencia de control debe equilibrar varias condiciones físicas.

Bibliografía

- [1] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [3] M. Mitchell, *An Introduction to Genetic Algorithms*. MIT Press, 1996.
- [4] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6.^a ed. Wiley, 2014.
- [5] Z. Michalewicz y D. B. Fogel, *How to Solve It: Modern Heuristics*, 2.^a ed. Springer, 2004.